



guide me step by step how to do this?

This is your RTU DIP392 Assignment 5 — due in about 1 day! You need to build an adaptive AI agent CLI app using the Google Gemini API, applying SOLID principles and GoF design patterns. Here's the complete step-by-step guide:^[1]

Step 1: Environment Setup

Install the required packages and get your API key:^[1]

```
pip install google-generativeai requests
```

Go to [Google AI Studio](#), generate a free Gemini API key, and set it as an environment variable:^[1]

```
# Linux/macOS
export GEMINI_API_KEY="your_api_key_here"

# Windows
set GEMINI_API_KEY="your_api_key_here"
```

Step 2: Project Structure

Separate your code into these files to satisfy the Single Responsibility Principle:^[1]

```
agent_project/
├── base_tool.py          # Abstract BaseTool interface
├── tool_registry.py      # ToolRegistry (Factory pattern)
├── memory_manager.py     # MemoryManager
├── tools/
│   ├── calculator_tool.py
│   ├── time_tool.py
│   ├── weather_tool.py   # custom
│   └── translator_tool.py # custom
├── agent.py              # Main Agent (ReAct loop)
└── main.py               # CLI entry point
```

Step 3: Implement the Core Architecture

base_tool.py — Strategy Pattern Interface

This is the abstract base class. Every tool must inherit from it:^[1]

```
from abc import ABC, abstractmethod
from typing import Any

class BaseTool(ABC):
    @abstractmethod
    def execute(self, **kwargs) -> Any:
        """Execute the tool logic."""
        pass

    @abstractmethod
    def get_declaration(self) -> dict:
        """Return the Gemini function calling JSON schema."""
        pass
```

tool_registry.py — Factory/Registry Pattern

Registers and dispatches tools dynamically without if-else chains:^[1]

```
from base_tool import BaseTool

class ToolRegistry:
    def __init__(self):
        self._tools: dict[str, BaseTool] = {}

    def register(self, name: str, tool: BaseTool):
        self._tools[name] = tool

    def execute(self, name: str, **kwargs):
        if name not in self._tools:
            raise ValueError(f"Unknown tool: {name}")
        return self._tools[name].execute(**kwargs)

    def get_declarations(self) -> list[dict]:
        return [tool.get_declaration() for tool in self._tools.values()]
```

memory_manager.py — Conversation History

Stores the chat turns within a session:^[1]

```
class MemoryManager:
    def __init__(self):
        self._history: list[dict] = []

    def add(self, role: str, content: str):
        self._history.append({"role": role, "parts": [content]})
```

```
def get_history(self) -> list[dict]:
    return self._history
```

Step 4: Implement the Four Tools

Built-in Tool 1 — CalculatorTool

```
from base_tool import BaseTool

class CalculatorTool(BaseTool):
    def execute(self, expression: str) -> str:
        try:
            return str(eval(expression, {"__builtins__": {}}))
        except Exception as e:
            return f"Error: {e}"

    def get_declaration(self) -> dict:
        return {
            "name": "calculator",
            "description": "Evaluates a math expression.",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {"type": "string", "description": "Math expression e.g. 2+2"},
                },
                "required": ["expression"]
            }
        }
```

Built-in Tool 2 — TimeTool

```
from datetime import datetime
from base_tool import BaseTool

class TimeTool(BaseTool):
    def execute(self) -> str:
        return datetime.now().strftime("Current date and time: %Y-%m-%d %H:%M:%S")

    def get_declaration(self) -> dict:
        return {
            "name": "get_time",
            "description": "Returns the current date and time.",
            "parameters": {"type": "object", "properties": {}}
        }
```

Custom Tool 1 — WeatherTool (uses wttr.in — no API key needed!)

```
import requests
from base_tool import BaseTool

class WeatherTool(BaseTool):
    def execute(self, city: str) -> str:
        try:
            r = requests.get(f"https://wttr.in/{city}?format=3", timeout=5)
            r.raise_for_status()
            return r.text
        except Exception as e:
            return f"Could not fetch weather for '{city}': {e}"

    def get_declaration(self) -> dict:
        return {
            "name": "get_weather",
            "description": "Gets the current weather for a given city.",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description": "City name e.g. 'Riga'"}
                },
                "required": ["city"]
            }
        }
```

Custom Tool 2 — TranslatorTool (uses MyMemory API — free)

```
import requests
from base_tool import BaseTool

class TranslatorTool(BaseTool):
    def execute(self, text: str, target_lang: str) -> str:
        try:
            r = requests.get(
                "https://api.mymemory.translated.net/get",
                params={"q": text, "langpair": f"en|{target_lang}"},
                timeout=5
            )
            return r.json()["responseData"]["translatedText"]
        except Exception as e:
            return f"Translation error: {e}"

    def get_declaration(self) -> dict:
        return {
            "name": "translate_text",
            "description": "Translates English text to a target language code (e.g. 'lv',",
            "parameters": {
                "type": "object",
                "properties": {
                    "text": {"type": "string", "description": "Text to translate"},
                    "target_lang": {"type": "string", "description": "Target language co"},
                },
            }
        }
```

```

        "required": ["text", "target_lang"]
    }
}

```

Step 5: Implement the Agent (ReAct Loop)

This is the core of the assignment — the Reason → Act → Observe loop:^[1]

```

import os
import google.generativeai as genai
from memory_manager import MemoryManager
from tool_registry import ToolRegistry

class Agent:
    def __init__(self, registry: ToolRegistry, memory: MemoryManager):
        genai.configure(api_key=os.environ["GEMINI_API_KEY"])
        self._registry = registry
        self._memory = memory
        self._model = genai.GenerativeModel(
            model_name="gemini-1.5-flash",
            tools=registry.get_declarations()
        )

    def run(self, user_input: str) -> str:
        self._memory.add("user", user_input)
        chat = self._model.start_chat(history=self._memory.get_history()[:-1])

        try:
            response = chat.send_message(user_input)

            # ReAct loop: keep acting until no more tool calls
            while response.candidates[0].content.parts[0].function_call.name:
                fc = response.candidates[0].content.parts[0].function_call
                tool_name = fc.name
                tool_args = dict(fc.args)

                try:
                    tool_result = self._registry.execute(tool_name, **tool_args)
                except Exception as e:
                    tool_result = f"Tool error: {e}"

                # Send tool result back to model (Observe step)
                response = chat.send_message(
                    genai.protos.Content(parts=[
                        genai.protos.Part(
                            function_response=genai.protos.FunctionResponse(
                                name=tool_name,
                                response={"result": tool_result}
                            )
                        )
                    ])
                )

        final_text = response.text

```

```
        self._memory.add("model", final_text)
        return final_text

    except Exception as e:
        return f"Agent error: {e}"
```

Step 6: Wire Everything in main.py

```
from tool_registry import ToolRegistry
from memory_manager import MemoryManager
from agent import Agent
from tools.calculator_tool import CalculatorTool
from tools.time_tool import TimeTool
from tools.weather_tool import WeatherTool
from tools.translator_tool import TranslatorTool

def main():
    registry = ToolRegistry()
    registry.register("calculator", CalculatorTool())
    registry.register("get_time", TimeTool())
    registry.register("get_weather", WeatherTool())
    registry.register("translate_text", TranslatorTool())

    memory = MemoryManager()
    agent = Agent(registry, memory)

    print("AI Agent ready. Type 'exit' to quit.\n")
    while True:
        user_input = input("You: ").strip()
        if user_input.lower() == "exit":
            break
        response = agent.run(user_input)
        print(f"Agent: {response}\n")

if __name__ == "__main__":
    main()
```

Step 7: Test Your Agent

Run with `python main.py` and test these scenarios to satisfy the evaluation criteria:^[1]

- "What is 345 * 78 + 99?" → triggers CalculatorTool
- "What time is it right now?" → triggers TimeTool
- "What's the weather in Riga?" → triggers WeatherTool
- "Translate 'Good morning' to Latvian" → triggers TranslatorTool
- "What's the weather in XYZNONEXISTENTCITY?" → tests error handling
- Ask a follow-up question to verify memory works across turns

Evaluation Checklist

Criterion	What to verify	Weight
Architecture & Patterns	BaseTool, ToolRegistry, MemoryManager, Agent are all separate classes	40%
Agent Functionality	ReAct loop runs correctly, memory persists across turns	30%
Custom Tools	WeatherTool + TranslatorTool have valid get_declaration() schemas	20%
Code Quality	Type hints, docstrings, error handling in every execute ()	10%

The most important thing to double-check is that your Agent class **never imports a specific tool directly** — all tool access must go through ToolRegistry. This is what proves your Open/Closed Principle compliance, which carries the heaviest weight at 40%.^[1]

✱

okay but lets do it in venv so my system env doesnt get affected

Smart move — using a venv is the right way to keep your project clean. Here's the complete setup:^[2]

Step 1: Create the Project Folder

```
mkdir ai_agent_a5
cd ai_agent_a5
```

Step 2: Create the Virtual Environment

```
python -m venv venv
```

This creates a venv/ folder inside your project — all packages go there, nothing touches your system Python.^[3]

Step 3: Activate the venv

Pick the command for your OS/shell:^[4]

OS	Shell	Command
Linux / macOS	bash/zsh	source venv/bin/activate
Windows	CMD	venv\Scripts\activate.bat

OS	Shell	Command
Windows	PowerShell	venv\Scripts\Activate.ps1

Once active, your terminal prompt will show (venv) at the start — that's your confirmation it's working.^[5]

Step 4: Install Packages Inside venv

```
pip install google-generativeai requests
```

Since the venv is active, this installs only inside the venv, not globally.^[6]

Step 5: Set Your API Key (within the same session)

Linux/macOS:

```
export GEMINI_API_KEY="your_api_key_here"
```

Windows CMD:

```
set GEMINI_API_KEY="your_api_key_here"
```

Windows PowerShell:

```
$env:GEMINI_API_KEY="your_api_key_here"
```

📌 **Tip:** To avoid re-setting the key every session, create a .env file and use python-dotenv. Install it with `pip install python-dotenv` and add this to the top of `main.py`:

```
from dotenv import load_dotenv
load_dotenv() # automatically reads .env file
```

Then your .env file just needs: `GEMINI_API_KEY=your_api_key_here`

Step 6: Run Your Agent

```
python main.py
```


Step 7: Deactivate When Done

```
deactivate
```

This drops you back to your normal system environment — nothing was modified globally.^[5]

Recommended Final Folder Structure

```
ai_agent_a5/
├── venv/                ← never commit this to Git
├── .env                 ← never commit this either
├── .gitignore
├── base_tool.py
├── tool_registry.py
├── memory_manager.py
├── agent.py
├── main.py
├── tools/
│   ├── calculator_tool.py
│   ├── time_tool.py
│   ├── weather_tool.py
│   └── translator_tool.py
```

Add a `.gitignore` with these two lines to protect your `venv` and API key if you push to GitHub:

```
venv/
.env
```



after step 5 you didnt give me any file to write or add please do it properly!

You're right, my bad! After setting up the `venv` and installing packages, here are all the files you need to create one by one:

File 1: `base_tool.py`

```
from abc import ABC, abstractmethod
from typing import Any

class BaseTool(ABC):
    @abstractmethod
    def execute(self, **kwargs) -> Any:
        pass

    @abstractmethod
```

```
def get_declaration(self) -> dict:
    pass
```

File 2: tool_registry.py

```
from base_tool import BaseTool

class ToolRegistry:
    def __init__(self):
        self._tools: dict[str, BaseTool] = {}

    def register(self, name: str, tool: BaseTool):
        self._tools[name] = tool

    def execute(self, name: str, **kwargs):
        if name not in self._tools:
            raise ValueError(f"Unknown tool: {name}")
        return self._tools[name].execute(**kwargs)

    def get_declarations(self) -> list[dict]:
        return [tool.get_declaration() for tool in self._tools.values()]
```

File 3: memory_manager.py

```
class MemoryManager:
    def __init__(self):
        self._history: list[dict] = []

    def add(self, role: str, content: str):
        self._history.append({"role": role, "parts": [content]})

    def get_history(self) -> list[dict]:
        return self._history
```

File 4: tools/calculator_tool.py

First, create the tools/ folder and an empty __init__.py inside it:

```
mkdir tools
touch tools/__init__.py      # Linux/macOS
# OR on Windows CMD:
type nul > tools\__init__.py
```

Then create the file:

```
from base_tool import BaseTool
```

```

class CalculatorTool(BaseTool):
    def execute(self, expression: str) -> str:
        try:
            return str(eval(expression, {"__builtins__": {}}))
        except Exception as e:
            return f"Calculation error: {e}"

    def get_declaration(self) -> dict:
        return {
            "name": "calculator",
            "description": "Evaluates a math expression like '2 + 2 * 5'.",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {
                        "type": "string",
                        "description": "A valid math expression e.g. '100 * 3 + 50'"
                    }
                },
                "required": ["expression"]
            }
        }

```

File 5: tools/time_tool.py

```

from datetime import datetime
from base_tool import BaseTool

class TimeTool(BaseTool):
    def execute(self, **kwargs) -> str:
        return datetime.now().strftime("Current date and time: %Y-%m-%d %H:%M:%S")

    def get_declaration(self) -> dict:
        return {
            "name": "get_time",
            "description": "Returns the current date and time.",
            "parameters": {
                "type": "object",
                "properties": {}
            }
        }

```

File 6: tools/weather_tool.py (*Custom Tool 1*)

```

import requests
from base_tool import BaseTool

class WeatherTool(BaseTool):
    def execute(self, city: str, **kwargs) -> str:
        try:
            r = requests.get(f"https://wttr.in/{city}?format=3", timeout=5)

```

```

        r.raise_for_status()
        return r.text
    except requests.exceptions.HTTPError:
        return f"Could not find weather for '{city}'. City may not exist."
    except Exception as e:
        return f"Weather fetch error: {e}"

def get_declaration(self) -> dict:
    return {
        "name": "get_weather",
        "description": "Gets the current weather for a given city.",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {
                    "type": "string",
                    "description": "Name of the city e.g. 'Riga', 'London'"
                }
            },
            "required": ["city"]
        }
    }

```

File 7: tools/translator_tool.py (*Custom Tool 2*)

```

import requests
from base_tool import BaseTool

class TranslatorTool(BaseTool):
    def execute(self, text: str, target_lang: str, **kwargs) -> str:
        try:
            r = requests.get(
                "https://api.mymemory.translated.net/get",
                params={"q": text, "langpair": f"en|{target_lang}"},
                timeout=5
            )
            r.raise_for_status()
            return r.json()["responseData"]["translatedText"]
        except Exception as e:
            return f"Translation error: {e}"

    def get_declaration(self) -> dict:
        return {
            "name": "translate_text",
            "description": "Translates English text to another language using a language",
            "parameters": {
                "type": "object",
                "properties": {
                    "text": {
                        "type": "string",
                        "description": "The English text to translate"
                    },
                    "target_lang": {
                        "type": "string",

```

```

        "description": "Target language code: 'lv' for Latvian, 'fr' for
    }
},
    "required": ["text", "target_lang"]
}
}

```

File 8: agent.py

```

import os
import google.generativeai as genai
from memory_manager import MemoryManager
from tool_registry import ToolRegistry

class Agent:
    def __init__(self, registry: ToolRegistry, memory: MemoryManager):
        genai.configure(api_key=os.environ["GEMINI_API_KEY"])
        self._registry = registry
        self._memory = memory
        self._model = genai.GenerativeModel(
            model_name="gemini-1.5-flash",
            tools=self._registry.get_declarations()
        )

    def run(self, user_input: str) -> str:
        self._memory.add("user", user_input)
        chat = self._model.start_chat(history=self._memory.get_history()[:-1])

        try:
            response = chat.send_message(user_input)

            # ReAct loop: Reason → Act → Observe
            while True:
                part = response.candidates[0].content.parts[0]
                if not hasattr(part, "function_call") or not part.function_call.name:
                    break

                fc = part.function_call
                tool_name = fc.name
                tool_args = dict(fc.args)

                try:
                    tool_result = self._registry.execute(tool_name, **tool_args)
                except Exception as e:
                    tool_result = f"Tool error: {e}"

                # Send observation back to model
                response = chat.send_message(
                    genai.protos.Content(parts=[
                        genai.protos.Part(
                            function_response=genai.protos.FunctionResponse(
                                name=tool_name,
                                response={"result": tool_result}
                            )
                        )
                    ])
                )

```

```

        )
    ])
)

final_text = response.text
self._memory.add("model", final_text)
return final_text

except Exception as e:
    return f"Agent error: {e}"

```

File 9: main.py

```

from dotenv import load_dotenv
load_dotenv()

from tool_registry import ToolRegistry
from memory_manager import MemoryManager
from agent import Agent
from tools.calculator_tool import CalculatorTool
from tools.time_tool import TimeTool
from tools.weather_tool import WeatherTool
from tools.translator_tool import TranslatorTool

def main():
    registry = ToolRegistry()
    registry.register("calculator", CalculatorTool())
    registry.register("get_time", TimeTool())
    registry.register("get_weather", WeatherTool())
    registry.register("translate_text", TranslatorTool())

    memory = MemoryManager()
    agent = Agent(registry, memory)

    print("== AI Personal Assistant ==")
    print("Type 'exit' to quit.\n")

    while True:
        user_input = input("You: ").strip()
        if not user_input:
            continue
        if user_input.lower() == "exit":
            print("Goodbye!")
            break
        response = agent.run(user_input)
        print(f"Agent: {response}\n")

if __name__ == "__main__":
    main()

```

File 10: .env

```
GEMINI_API_KEY=your_actual_api_key_here
```

Also install `python-dotenv` so `main.py` can read it:

```
pip install python-dotenv
```

File 11: .gitignore

```
venv/  
.env  
__pycache__/  
*.pyc
```

Final Folder Check

Your project should look exactly like this before running:

```
ai_agent_a5/  
├─ venv/  
├─ .env  
├─ .gitignore  
├─ base_tool.py  
├─ tool_registry.py  
├─ memory_manager.py  
├─ agent.py  
├─ main.py  
└─ tools/  
    ├─ __init__.py  
    ├─ calculator_tool.py  
    ├─ time_tool.py  
    ├─ weather_tool.py  
    └─ translator_tool.py
```

Once all files are in place, activate `venv` and run:

```
source venv/bin/activate    # or Windows equivalent  
python main.py
```

gimme powershell code to do this!

Here's a single PowerShell script you can copy-paste and run all at once. It creates every folder, every file, and writes all the code inside them:

```
# — Create project folder and navigate into it —
New-Item -ItemType Directory -Name "ai_agent_a5" | Out-Null
Set-Location "ai_agent_a5"

# — Create virtual environment and activate it —
python -m venv venv
.\venv\Scripts\Activate.ps1

# — Install dependencies —
pip install google-generativeai requests python-dotenv

# — Create tools subfolder —
New-Item -ItemType Directory -Name "tools" | Out-Null

# =====
# Write all project files
# =====

# .env
@"
GEMINI_API_KEY=your_actual_api_key_here
"@ | Set-Content .env

# .gitignore
@"
venv/
.env
__pycache__/
*.pyc
"@ | Set-Content .gitignore

# base_tool.py
@"
from abc import ABC, abstractmethod
from typing import Any

class BaseTool(ABC):
    @abstractmethod
    def execute(self, **kwargs) -> Any:
        pass

    @abstractmethod
    def get_declaration(self) -> dict:
        pass
"@ | Set-Content base_tool.py

# tool_registry.py
@"
from base_tool import BaseTool
```



```

class ToolRegistry:
    def __init__(self):
        self._tools: dict[str, BaseTool] = {}

    def register(self, name: str, tool: BaseTool):
        self._tools[name] = tool

    def execute(self, name: str, **kwargs):
        if name not in self._tools:
            raise ValueError(f"Unknown tool: {name}")
        return self._tools[name].execute(**kwargs)

    def get_declarations(self) -> list[dict]:
        return [tool.get_declaration() for tool in self._tools.values()]
"@ | Set-Content tool_registry.py

# memory_manager.py
@"
class MemoryManager:
    def __init__(self):
        self._history: list[dict] = []

    def add(self, role: str, content: str):
        self._history.append({"role": role, "parts": [content]})

    def get_history(self) -> list[dict]:
        return self._history
"@ | Set-Content memory_manager.py

# agent.py
@"
import os
import google.generativeai as genai
from memory_manager import MemoryManager
from tool_registry import ToolRegistry

class Agent:
    def __init__(self, registry: ToolRegistry, memory: MemoryManager):
        genai.configure(api_key=os.environ["GEMINI_API_KEY"])
        self._registry = registry
        self._memory = memory
        self._model = genai.GenerativeModel(
            model_name="gemini-1.5-flash",
            tools=self._registry.get_declarations()
        )

    def run(self, user_input: str) -> str:
        self._memory.add("user", user_input)
        chat = self._model.start_chat(history=self._memory.get_history()[::-1])

        try:
            response = chat.send_message(user_input)

            while True:
                part = response.candidates[0].content.parts[0]
                if not hasattr(part, "function_call") or not part.function_call.name:

```

```

        break

    fc = part.function_call
    tool_name = fc.name
    tool_args = dict(fc.args)

    try:
        tool_result = self._registry.execute(tool_name, **tool_args)
    except Exception as e:
        tool_result = f"Tool error: {e}"

    response = chat.send_message(
        genai.protos.Content(parts=[
            genai.protos.Part(
                function_response=genai.protos.FunctionResponse(
                    name=tool_name,
                    response={"result": tool_result}
                )
            )
        ])
    )

    final_text = response.text
    self._memory.add("model", final_text)
    return final_text

except Exception as e:
    return f"Agent error: {e}"
"@ | Set-Content agent.py

# main.py
@"
from dotenv import load_dotenv
load_dotenv()

from tool_registry import ToolRegistry
from memory_manager import MemoryManager
from agent import Agent
from tools.calculator_tool import CalculatorTool
from tools.time_tool import TimeTool
from tools.weather_tool import WeatherTool
from tools.translator_tool import TranslatorTool

def main():
    registry = ToolRegistry()
    registry.register("calculator", CalculatorTool())
    registry.register("get_time", TimeTool())
    registry.register("get_weather", WeatherTool())
    registry.register("translate_text", TranslatorTool())

    memory = MemoryManager()
    agent = Agent(registry, memory)

    print("== AI Personal Assistant ==")
    print("Type 'exit' to quit.\n")

```

```

while True:
    user_input = input("You: ").strip()
    if not user_input:
        continue
    if user_input.lower() == "exit":
        print("Goodbye!")
        break
    response = agent.run(user_input)
    print(f"Agent: {response}\n")

if __name__ == "__main__":
    main()
"@ | Set-Content main.py

# tools\__init__.py
@"
"@ | Set-Content tools\__init__.py

# tools\calculator_tool.py
@"
from base_tool import BaseTool

class CalculatorTool(BaseTool):
    def execute(self, expression: str, **kwargs) -> str:
        try:
            return str(eval(expression, {"__builtins__": {}}))
        except Exception as e:
            return f"Calculation error: {e}"

    def get_declaration(self) -> dict:
        return {
            "name": "calculator",
            "description": "Evaluates a math expression like '2 + 2 * 5'.",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {
                        "type": "string",
                        "description": "A valid math expression e.g. '100 * 3 + 50'"
                    }
                },
                "required": ["expression"]
            }
        }
"@ | Set-Content tools\calculator_tool.py

# tools\time_tool.py
@"
from datetime import datetime
from base_tool import BaseTool

class TimeTool(BaseTool):
    def execute(self, **kwargs) -> str:
        return datetime.now().strftime("Current date and time: %Y-%m-%d %H:%M:%S")

    def get_declaration(self) -> dict:

```

```

        return {
            "name": "get_time",
            "description": "Returns the current date and time.",
            "parameters": {
                "type": "object",
                "properties": {}
            }
        }
    }

"@ | Set-Content tools\time_tool.py

# tools\weather_tool.py
@"
import requests
from base_tool import BaseTool

class WeatherTool(BaseTool):
    def execute(self, city: str, **kwargs) -> str:
        try:
            r = requests.get(f"https://wttr.in/{city}?format=3", timeout=5)
            r.raise_for_status()
            return r.text
        except requests.exceptions.HTTPError:
            return f"Could not find weather for '{city}'. City may not exist."
        except Exception as e:
            return f"Weather fetch error: {e}"

    def get_declaration(self) -> dict:
        return {
            "name": "get_weather",
            "description": "Gets the current weather for a given city.",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {
                        "type": "string",
                        "description": "Name of the city e.g. 'Riga', 'London'"
                    }
                },
            },
            "required": ["city"]
        }
    }

"@ | Set-Content tools\weather_tool.py

# tools\translator_tool.py
@"
import requests
from base_tool import BaseTool

class TranslatorTool(BaseTool):
    def execute(self, text: str, target_lang: str, **kwargs) -> str:
        try:
            r = requests.get(
                "https://api.mymemory.translated.net/get",
                params={"q": text, "langpair": f"en|{target_lang}"},
                timeout=5
            )

```

```

        r.raise_for_status()
        return r.json()["responseData"]["translatedText"]
    except Exception as e:
        return f"Translation error: {e}"

def get_declaration(self) -> dict:
    return {
        "name": "translate_text",
        "description": "Translates English text to another language using a language",
        "parameters": {
            "type": "object",
            "properties": {
                "text": {
                    "type": "string",
                    "description": "The English text to translate"
                },
                "target_lang": {
                    "type": "string",
                    "description": "Target language code: 'lv' Latvian, 'fr' French,"
                }
            },
            "required": ["text", "target_lang"]
        }
    }

"@ | Set-Content tools\translator_tool.py

# — Done! —
Write-Host ""
Write-Host "✔ Project created successfully!" -ForegroundColor Green
Write-Host "📝 Don't forget to edit .env and replace 'your_actual_api_key_here'" -ForegroundColor Green
Write-Host "🚀 Run with: python main.py" -ForegroundColor Cyan

```

⚠ One Thing Before You Run

If PowerShell blocks script execution, run this once first:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Then paste and run the big script above. After it finishes:

1. Open .env and replace your_actual_api_key_here with your real Gemini API key
2. Run your agent:

```
python main.py
```

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! <https://aka.ms/PSWindows>

PS D:\5Sem\ACS\ai_agent_a5\ai_agent_a5> @"

```

import requests
from base_tool import BaseTool
class TranslatorTool(BaseTool):
def execute(self, text:str, target_lang:str, **kwargs) -> str:
try:
r=requests.get(
"https://api.mymemory.translated.net/get",
params={"q":text, "langpair":f"en|{target_lang}"},
timeout=5
)
rraise_for_status()
return r.json()["responseData"]["translatedText"]
except Exception as e:
return f"Translation error:{e}"

def get_declaration(self) -> dict:
return {
"name": "translate_text",
"description": "Translates English text to another language using a",
"parameters": {
"type": "object",
"properties": {
"text": {
"type": "string",
"description": "The English text to translate"
},
"target_lang": {
"type": "string",
"description": "Target language code: 'lv' Latvian, 'fr"
}
},
"required": ["text", "target_lang"]
}
}

```

"@ | Set-Content tools\translator_tool.py
PS D:\5Sem\ACS\ai_agent_a5\ai_agent_a5>@"

```

import requests
from base_tool import BaseTool
class WeatherTool(BaseTool):
def execute(self, city: str, **kwargs) -> str:
try:
r = requests.get(f"https://wttr.in/{city}?format=3", timeout=5)
r.raise_for_status()
return r.text
except requests.exceptions.HTTPError:
return f"Could not find weather for '{city}'. City may not exist."
except Exception as e:
return f"Weather fetch error: {e}"

```

```

def get_declaration(self) -> dict:
return {
    "name": "get_weather",
    "description": "Gets the current weather for a given city.",
    "parameters": {
        "type": "object",
        "properties": {
            "city": {
                "type": "string",
                "description": "Name of the city e.g. 'Riga', 'London'"
            }
        },
        "required": ["city"]
    }
}

```

"@ | Set-Content tools\weather_tool.py

PS D:\5Sem\ACS\ai_agent_a5\ai_agent_a5> @"


```

from dotenv import load_dotenv
load_dotenv()
from tool_registry import ToolRegistry
from memory_manager import MemoryManager
from agent import Agent
from tools.calculator_tool import CalculatorTool
from tools.time_tool import TimeTool
from tools.weather_tool import WeatherTool
from tools.translator_tool import TranslatorTool
def main():
    registry = ToolRegistry()
    registry.register("calculator", CalculatorTool())
    registry.register("get_time", TimeTool())
    registry.register("get_weather", WeatherTool())
    registry.register("translate_text", TranslatorTool())

```

```

memory = MemoryManager()
agent = Agent(registry, memory)

print("== AI Personal Assistant ==")
print("Type 'exit' to quit.\n")

while True:
    user_input = input("You: ").strip()
    if not user_input:
        continue
    if user_input.lower() == "exit":
        print("Goodbye!")
        break
    response = agent.run(user_input)
    print(f"Agent: {response}\n")

```

```
if name == "main":
```

```
main()
```

```
"@ | Set-Content main.py
```

```
PS D:\5Sem\ACS\ai_agent_a5\ai_agent_a5> .venv\Scripts\Activate.ps1 tenv
```

```
(venv) PS D:\5Sem\ACS\ai_agent_a5\ai_agent_a5> python main.py
```

```
D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\agent.py:2: FutureWarning:
```

All support for the `google.generativeai` package has ended. It will no longer be receiving updates or bug fixes. Please switch to the `google.genai` package as soon as possible.

See README for more details:

<https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md>

```
import google.generativeai as genai
```

```
Traceback (most recent call last):
```

```
File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\venv\Lib\site-packages\proto\marshal\rules\message.py", line 36, in to_proto
```

```
return self._descriptor(**value)
```

~~~~~^

ValueError: Protocol message Schema has no "type" field.

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

File "D:\5Sem\ACS\ai\_agent\_a5\ai\_agent\_a5\main.py", line 36, in <module>  
main()

~~~~^

File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\main.py", line 20, in main
agent = Agent(registry, memory)

File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\agent.py", line 11, in init
self._model = genai.GenerativeModel(
~~~~~^

model\_name="gemini-1.5-flash",  
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

tools=self.\_registry.get\_declarations()  
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

)  
^

File "D:\5Sem\ACS\ai\_agent\_a5\ai\_agent\_a5\venv\Lib\site-  
packages\google\generativeai\generative\_models.py", line 87, in init  
self.\_tools = content\_types.to\_function\_library(tools)

~~~~~^

File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\venv\Lib\site-
packages\google\generativeai\types\content_types.py", line 917, in to_function_library
return FunctionLibrary(tools=lib)

File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\venv\Lib\site-
packages\google\generativeai\types\content_types.py", line 855, in init
tools = _make_tools(tools)

File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\venv\Lib\site-
packages\google\generativeai\types\content_types.py", line 898, in _make_tools
tools = [_make_tool(t) for t in tools]
~~~~~^^

File "D:\5Sem\ACS\ai\_agent\_a5\ai\_agent\_a5\venv\Lib\site-  
packages\google\generativeai\types\content\_types.py", line 823, in \_make\_tool  
return Tool(function\_declarations=[protos.FunctionDeclaration(\*\*fd)])  
~~~~~^

File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\venv\Lib\site-packages\proto\message.py", line
677, in init

pb_value = marshal.to_proto(pb_type, value)

File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\venv\Lib\site-packages\proto\marshal\marshal.py",
line 233, in to_proto

pb_value = self.get_rule(proto_type=proto_type).to_proto(value)

File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\venv\Lib\site-packages\proto\marshal\rules\
message.py", line 46, in to_proto

```

return self._wrapper(value)._pb
~~~~~^^^^^^
File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\venv\Lib\site-packages\proto\message.py", line
677, in init
pb_value = marshal.to_proto(pb_type, value)
File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\venv\Lib\site-packages\proto\marshal\marshal.py",
line 233, in to_proto
pb_value = self.get_rule(proto_type=proto_type).to_proto(value)
File "D:\5Sem\ACS\ai_agent_a5\ai_agent_a5\venv\Lib\site-packages\proto\marshal\rules\
enums.py", line 56, in to_proto
return self._enum[value].value
~~~~~^^^^^^
File "C:\Python314\Lib\enum.py", line 774, in getitem
return cls._member_map[name]
~~~~~^^^^^^
KeyError: 'object'
(venv) PS D:\5Sem\ACS\ai_agent_a5\ai_agent_a5>

```

Two problems here:^[17]

1. `google.generativeai` is **fully deprecated** — must switch to `google-genai`
2. Schema types must be UPPERCASE ("OBJECT", "STRING") in the new SDK

Here's exactly what to run and replace:

Step 1: Swap the Package

```

pip uninstall google-generativeai -y
pip install google-genai

```

Step 2: Rewrite `agent.py`

The entire API structure changed. Run this in PowerShell:

```

@
import os
from google import genai
from google.genai import types
from memory_manager import MemoryManager
from tool_registry import ToolRegistry

class Agent:
    def __init__(self, registry: ToolRegistry, memory: MemoryManager):
        self._client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
        self._registry = registry
        self._memory = memory
        self._tool = types.Tool(function_declarations=registry.get_declarations())

```

```

        self._config = types.GenerateContentConfig(tools=[self._tool])

def run(self, user_input: str) -> str:
    chat = self._client.chats.create(
        model="gemini-2.0-flash",
        config=self._config,
        history=self._memory.get_history()
    )

    try:
        response = chat.send_message(user_input)

        # ReAct loop: Reason -> Act -> Observe
        while response.function_calls:
            tool_response_parts = []
            for fc in response.function_calls:
                try:
                    result = self._registry.execute(fc.name, **fc.args)
                except Exception as e:
                    result = f"Tool error: {e}"

                tool_response_parts.append(
                    types.Part.from_function_response(
                        name=fc.name,
                        response={"result": str(result)}
                    )
                )
            response = chat.send_message(tool_response_parts)

        final_text = response.text
        self._memory.set_history(chat.get_history())
        return final_text

    except Exception as e:
        return f"Agent error: {e}"
"@ | Set-Content agent.py

```

Step 3: Update memory_manager.py

The new SDK stores history as types.Content objects, so MemoryManager needs a set_history() method:

```

@"
class MemoryManager:
    def __init__(self):
        self._history = []

    def get_history(self) -> list:
        return self._history

    def set_history(self, history: list):
        self._history = history
"@ | Set-Content memory_manager.py

```

Step 4: Fix All Tool Schema Types to UPPERCASE

The new SDK requires "OBJECT", "STRING" etc.in uppercase. Rewrite all 4 tool files:^[18]

```
@  
from base_tool import BaseTool  
  
class CalculatorTool(BaseTool):  
    def execute(self, expression: str, **kwargs) -> str:  
        try:  
            return str(eval(expression, {"__builtins__": {}}))  
        except Exception as e:  
            return f"Calculation error: {e}"  
  
    def get_declaration(self) -> dict:  
        return {  
            "name": "calculator",  
            "description": "Evaluates a math expression like '2 + 2 * 5'.",  
            "parameters": {  
                "type": "OBJECT",  
                "properties": {  
                    "expression": {  
                        "type": "STRING",  
                        "description": "A valid math expression e.g. '100 * 3 + 50'"  
                    }  
                },  
                "required": ["expression"]  
            }  
        }  
"@ | Set-Content tools\calculator_tool.py
```

```
@  
from datetime import datetime  
from base_tool import BaseTool  
  
class TimeTool(BaseTool):  
    def execute(self, **kwargs) -> str:  
        return datetime.now().strftime("Current date and time: %Y-%m-%d %H:%M:%S")  
  
    def get_declaration(self) -> dict:  
        return {  
            "name": "get_time",  
            "description": "Returns the current date and time.",  
            "parameters": {  
                "type": "OBJECT",  
                "properties": {}  
            }  
        }  
"@ | Set-Content tools\time_tool.py
```

```
@  
import requests
```

```

from base_tool import BaseTool

class WeatherTool(BaseTool):
    def execute(self, city: str, **kwargs) -> str:
        try:
            r = requests.get(f"https://wttr.in/{city}?format=3", timeout=5)
            r.raise_for_status()
            return r.text
        except requests.exceptions.HTTPError:
            return f"Could not find weather for '{city}'. City may not exist."
        except Exception as e:
            return f"Weather fetch error: {e}"

    def get_declaration(self) -> dict:
        return {
            "name": "get_weather",
            "description": "Gets the current weather for a given city.",
            "parameters": {
                "type": "OBJECT",
                "properties": {
                    "city": {
                        "type": "STRING",
                        "description": "Name of the city e.g. 'Riga', 'London'"
                    }
                },
                "required": ["city"]
            }
        }

"@ | Set-Content tools\weather_tool.py

```

```

@"
import requests
from base_tool import BaseTool

class TranslatorTool(BaseTool):
    def execute(self, text: str, target_lang: str, **kwargs) -> str:
        try:
            r = requests.get(
                "https://api.mymemory.translated.net/get",
                params={"q": text, "langpair": f"en|{target_lang}"},
                timeout=5
            )
            r.raise_for_status()
            return r.json()["responseData"]["translatedText"]
        except Exception as e:
            return f"Translation error: {e}"

    def get_declaration(self) -> dict:
        return {
            "name": "translate_text",
            "description": "Translates English text to another language using a language",
            "parameters": {
                "type": "OBJECT",
                "properties": {
                    "text": {

```

```

        "type": "STRING",
        "description": "The English text to translate"
    },
    "target_lang": {
        "type": "STRING",
        "description": "Target language code: 'lv' Latvian, 'fr' French,
    }
},
"required": ["text", "target_lang"]
}
}
"@ | Set-Content tools\translator_tool.py

```

Step 5: Run Again

```
python main.py
```

You should see `== AI Personal Assistant ==` with no errors. The key changes were: new `google-genai` package, new `genai.Client()` pattern, and uppercase schema types throughout. [\[17\]](#) [\[18\]](#)

✱

okay now tell me how to use?

Once `python main.py` is running and you see `== AI Personal Assistant ==`, just type naturally. Here's what each tool can handle:

Calculator

Ask any math question and the agent will call the `CalculatorTool` automatically:

```

You: What is 345 * 78 + 99?
You: Calculate (500 / 4) ^ 2
You: What's 15% of 2400?

```

Current Time

```

You: What time is it right now?
You: What is today's date?

```

Weather (*Custom Tool 1*)

You: What's the weather in Riga?
You: Is it raining in London right now?
You: What's the temperature in Tokyo?

Translator (*Custom Tool 2*)

You: Translate "Good morning" to Latvian
You: How do you say "thank you" in Japanese?
You: Translate "I love programming" to French

Language codes the tool understands: lv (Latvian), fr (French), de (German), es (Spanish), ja (Japanese), ru (Russian), it (Italian)

Memory — Multi-turn Conversation

The agent remembers the whole session, so you can do follow-ups naturally:

You: What's the weather in Paris?
Agent: Paris: ☀ +12°C

You: Should I bring a jacket?
Agent: Yes, 12°C is cool enough that a light jacket would be comfortable.

Combining Multiple Tools

You can ask something that needs more than one tool in a single message:

You: What time is it and what's the weather in Riga?
You: Translate the current weather in Berlin to Latvian

Error Handling — Testing Robustness

These are good to test before submission since error handling is part of the grading criteria:

You: What's the weather in XYZFAKECITY123?
You: Calculate 10 / 0
You: Translate "hello" to xx ← invalid language code

Exiting

You: exit

☒ **Tip for your submission:** Run through at least one example of each tool and one multi-turn conversation — that directly covers the Agent Functionality (30%) and Custom Tool (20%) grading criteria from your assignment.

```
(venv) PS D:\5Sem\ACS\ai_agent_a5\ai_agent_a5>
python main.py
```

```
=== AI Personal Assistant ===
```

```
Type 'exit' to quit.
```

You: what is possibility of world war 3?

Agent: Agent error: 429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google.dev/gemini-api/docs/rate-limits>. To monitor your current usage, head to: <https://ai.dev/rate-limit>.\n* Quota exceeded for metric: [generativelanguage.googleapis.com/generate_content_free_tier_input_token_count](https://ai.google.dev/gemini-api/docs/rate-limits), limit: 0, model: gemini-2.0-flash\n* Quota exceeded for metric: [generativelanguage.googleapis.com/generate_content_free_tier_requests](https://ai.google.dev/gemini-api/docs/rate-limits), limit: 0, model: gemini-2.0-flash\n* Quota exceeded for metric: [generativelanguage.googleapis.com/generate_content_free_tier_requests](https://ai.google.dev/gemini-api/docs/rate-limits), limit: 0, model: gemini-2.0-flash\nPlease retry in 11.497764196s, 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': '<https://ai.google.dev/gemini-api/docs/rate-limits>'}], '@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': '[generativelanguage.googleapis.com/generate_content_free_tier_input_token_count](https://ai.google.dev/gemini-api/docs/rate-limits)', 'quotaId': 'GenerateContentInputTokensPerModelPerMinute-FreeTier', 'quotaDimensions': {'location': 'global', 'model': 'gemini-2.0-flash'}}, {'quotaMetric': '[generativelanguage.googleapis.com/generate_content_free_tier_requests](https://ai.google.dev/gemini-api/docs/rate-limits)', 'quotaId': 'GenerateRequestsPerMinutePerProjectPerModel-FreeTier', 'quotaDimensions': {'location': 'global', 'model': 'gemini-2.0-flash'}}, {'quotaMetric': '[generativelanguage.googleapis.com/generate_content_free_tier_requests](https://ai.google.dev/gemini-api/docs/rate-limits)', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'location': 'global', 'model': 'gemini-2.0-flash'}}], '@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '11s']}]}

You: 365 * 2

Agent: Agent error: 429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google.dev/gemini-api/docs/rate-limits>. To monitor your current usage, head to: <https://ai.dev/rate-limit>.\n* Quota exceeded for metric: [generativelanguage.googleapis.com/generate_content_free_tier_input_token_count](https://ai.google.dev/gemini-api/docs/rate-limits), limit: 0, model: gemini-2.0-flash\n* Quota exceeded for metric: [generativelanguage.googleapis.com/generate_content_free_tier_requests](https://ai.google.dev/gemini-api/docs/rate-limits),

limit:0, model:gemini-2.0-flash\n* Quota exceeded for metric: [generativelanguage.googleapis.com/generate_content_free_tier_requests](https://ai.google.dev/gemini-api/docs/rate-limits), limit:0, model:gemini-2.0-flash\nPlease retry in 140645069s', 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_input_token_count', 'quotaId': 'GenerateContentInputTokensPerModelPerMinute-FreeTier', 'quotaDimensions': {'model': 'gemini-2.0-flash', 'location': 'global'}}, {'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerMinutePerProjectPerModel-FreeTier', 'quotaDimensions': {'model': 'gemini-2.0-flash', 'location': 'global'}}, {'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'model': 'gemini-2.0-flash', 'location': 'global'}}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '1s'}]}}

You:

Your API key is fine — the problem is that `gemini-2.0-flash` was retired on March 3, 2026 and its free tier quota is now literally `limit: 0`. You just need to switch to a model that's still active on the free tier.^[32]

Quick Fix — One Line Change in `agent.py`

Run this in PowerShell to update the model name:

```
(Get-Content agent.py) -replace 'gemini-2.0-flash', 'gemini-2.5-flash' | Set-Content agent.py
```

That's it. Then run again:

```
python main.py
```

Why This Happened

Google retired `gemini-2.0-flash` on March 3, 2026, setting its free tier to `limit: 0`. Here are the currently working free tier models:^[32]

| Model | Requests/min | Requests/day | Best For |
|-----------------------|--------------|--------------|---------------------|
| gemini-2.5-flash | 10 | 500 | Your assignment ✓ |
| gemini-2.5-flash-lite | 15 | 1,000 | High volume testing |
| gemini-2.5-pro | 5 | 100 | Complex reasoning |

`gemini-2.5-flash` is the best pick for your assignment — it's the current default, has the highest daily quota among capable models, and is more than enough for an academic project.^[32]

△ Free tier resets at midnight Pacific Time (UTC-8). So if you somehow hit the 500/day limit during heavy testing, just wait for the reset.

✱

okay so is that all?

Yes, that single model name change is all you need! Just run `python main.py` and it should work.

Before you submit though, quickly go through this checklist since your deadline is **tomorrow** (March 30):^[47]

Pre-Submission Checklist

Test each tool works:

- ☐ Ask a math question → CalculatorTool fires
- ☐ Ask the time/date → TimeTool fires
- ☐ Ask weather for a real city → WeatherTool fires
- ☐ Ask to translate something → TranslatorTool fires

Test memory:

- ☐ Ask a follow-up question referencing something from earlier in the same session

Test error handling (graded!):

- ☐ Weather for a fake city → graceful error message, agent doesn't crash
- ☐ Invalid language code for translator → graceful error, agent continues

Check your architecture (worth 40%):

- ☐ Agent class never directly imports any tool — only uses ToolRegistry
- ☐ All 4 files are separate: `base_tool.py`, `tool_registry.py`, `memory_manager.py`, `agent.py`

Once all boxes are checked, zip the project folder (excluding `venv/` and `.env`) and submit. Good luck! ☒

✱

okay gimme testing questions

Here are all the test prompts to type into your agent, organized by what they're testing:

☒ Calculator Tool

What is $365 * 24$?
Calculate $(150 + 250) * 2 - 80$
What is 17% of 8500?
What is the square root of 144?

☒ Time Tool

What is the current date and time?
What day is it today?

☒ Weather Tool (Custom 1)

What is the weather in Riga?
What is the temperature in Berlin right now?
Is it cold in Tokyo today?

☒ Translator Tool (Custom 2)

Translate "Good morning" to Latvian
How do you say "thank you very much" in French?
Translate "I love artificial intelligence" to German
Translate "Hello" to Japanese

☒ Multi-Tool (tests agent reasoning)

What time is it and what is the weather in London?
Translate the word "sunny" to Latvian, then tell me the weather in Riga
What is $100 * 365$ and also what is the weather in Paris?

☒ Memory (ask these back to back)

You: What is the weather in Riga?
You: Is that warmer or colder than usual for this time of year?
You: Translate that city name to Russian

☒ Error Handling (must not crash!)

```
What is the weather in XYZFAKECITY999?  
Translate "hello" to zz  
Calculate 10 / 0  
What is the weather in ??###!!!?
```

Run all of these before submitting — if the agent responds gracefully to every single one without crashing, you're good to go! ✓

Agent: I could not find the weather for 'XYZFAKECITY999'. The city may not exist.

'ZZ' is an invalid target language. Please use a 2-letter ISO or RFC3066 code, like ZH-CN.

I encountered a calculation error: division by zero.

I could not find the weather for '??###!!!'. The city may not exist.

That's perfect — your error handling is working exactly as it should! ☒

Every error case responded gracefully without crashing:

| Test | Result |
|--------------------------|-------------------------------|
| Fake city weather | ✓ Friendly message, continued |
| Invalid language code zz | ✓ Explained the issue clearly |
| Division by zero | ✓ Caught the exception |
| Garbage input weather | ✓ Handled cleanly |

This directly satisfies the **Code Quality & Error Handling (10%)** grading criterion. Now just quickly confirm the other tools worked fine too — if they did, you're fully done and ready to zip and submit! ☒^[48]

✱

1. A5.pdf
2. <https://hostagenda.com/blog/install-and-import-google-generativeai>
3. <https://docs.python.org/3/library/venv.html>
4. <https://www.geeksforgeeks.org/python/create-virtual-environment-using-venv-python/>
5. <https://python.land/virtual-environments/virtualenv>
6. <https://www.goldyarora.com/blog/python-virtual-environments>
7. <https://googleapis.dev/python/generativelanguage/latest/>
8. <https://chattingwithmodels.wordpress.com/2025/04/06/a-beginners-guide-to-the-google-gen-ai-sdk-for-python/>
9. <https://www.geeksforgeeks.org/python/creating-python-virtual-environment-windows-linux/>

10. <https://www.youtube.com/watch?v=ORjE4gxL6pE>
11. <https://www.linkedin.com/pulse/guide-setting-up-python-anaconda-visual-studio-code-googles-hassan-qabtf>
12. <https://docs.cloud.google.com/vertex-ai/docs/start/install-sdk>
13. <https://gist.github.com/allenthomas01/09fb906a4b23e88db6cb6779273f6cf8>
14. <https://reelmind.ai/blog/activate-venv-from-python-script-automate-development>
15. <https://stackoverflow.com/questions/14604699/how-can-i-activate-a-virtualenv-in-linux>
16. <https://stackoverflow.com/questions/74966861/activating-python-virtual-environment-on-windows-11>
17. <https://ai.google.dev/gemini-api/docs/function-calling>
18. <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/model-reference/function-calling>
19. <https://github.com/langchain-ai/langchain-google/issues/711>
20. <https://geminibyexample.com/021-tool-use-function-calling/>
21. <https://github.com/google-gemini/deprecated-generative-ai-python/commit/7d2c17e4738316455212526c4f909ce668b62676>
22. <https://ai.google.dev/gemini-api/docs/interactions>
23. <https://stackoverflow.com/questions/78090031/google-generative-ai-call-function-doesnt-work/78092721>
24. <https://github.com/AlexanderJiazx/genai-doc>
25. <https://github.com/googleapis/python-genai>
26. <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/function-calling>
27. <https://www.philschmid.de/gemini-function-calling>
28. <https://ai.google.dev/gemini-api/docs/function-calling/tutorial?lang=python>
29. <https://googleapis.github.io/python-genai/>
30. <https://github.com/google-gemini/deprecated-generative-ai-python/commit/d894807aa33aaff8434200bd23aad83e28f47051>
31. https://atamel.dev/posts/2025/04-08_simplified_function_calling_gemini/
32. <https://yingtu.ai/en/blog/gemini-api-free-tier>
33. <https://summarizemeeting.com/en/blog/is-gemini-api-free-complete-2025-pricing-guide-free-tier-rate-limits-and-cost-comparison>
34. <https://github.com/google-gemini/gemini-cli/issues/8883>
35. <https://free-llm.com/compare/google-gemini-2-0-flash-free-vs-gemini-1-5-pro>
36. <https://stackoverflow.com/questions/78535824/why-does-my-code-send-429-quota-exceeds-on-google-gemini-api-with-generative-ui>
37. <https://ai.google.dev/gemini-api/docs/rate-limits>
38. <https://help.apify.com/en/ai-studio-free-quota-exhausted-continue-use-gemini-guide-en.html>
39. <https://yingtu.ai/en/blog/google-gemini-api-free-tier-limits-2025>
40. <https://discuss.ai.google.dev/t/gemini-api-free-tier-limit-is-0-cannot-use-api-despite-valid-key/114215>

- 41. <https://discuss.ai.google.dev/t/my-paid-account-is-treated-as-free-causing-gemini-api-quota-exceeded-errors/106053>
- 42. https://www.reddit.com/r/Bard/comments/1pgg87t/api_error_you_have_exhausted_your_daily_quota_on/
- 43. <https://www.aifreeapi.com/en/posts/gemini-api-free-tier-not-working>
- 44. <https://yingtu.ai/en/blog/google-gemini-api-free-tier>
- 45. <https://yingtu.ai/blog/gemini-api-quota-exceeded-fix>
- 46. <https://github.com/google-gemini/gemini-cli/issues/14986>
- 47. A5.pdf
- 48. A5.pdf